LEVELUP CLOUD TRAININGS

levelupedu.net | Trainer: Sateesh Pabbathi — MCT & Microsoft MVP

CLASS NOTES

PySpark — Resilient Distributed Datasets (RDD)

Session 01 | Data Engineering Masterclass

Session Topics	RDD Creation • map() • flatMap() • reduceByKey() • sortByKey() • sortBy() • collect() • take()
Pre-requisite	Basic Python — lists, lambda functions, tuples
Platform	Google Colab	PySpark 3.2.1	Java 8
Course Module	Data Engineering Masterclass — PySpark Foundation

"Think of every concept like a real factory on the shop floor — not just code on a screen."

Section 1 — What is an RDD?

The Big Picture — Why RDD Exists

Before we write a single line of code, let's understand the problem PySpark solves. Imagine you work at Zomato and you need to analyse 1 crore order records. You can't fit that data in one computer's memory. So what do you do?

> **Real-World Analogy:** Think of Zomato's massive order log as a huge pile of delivery receipts. Instead of one person reading all 1 crore receipts, you divide the pile across 1,000 workers (computers). Each worker reads their chunk, processes it, and returns results. That is exactly what Spark does — and an RDD is how it represents that distributed pile of data.

Full Form & Definition

Term	Meaning
`R — Resilient`	Fault-tolerant. If one computer crashes, Spark knows how to recompute the lost data automatically.
`D — Distributed`	Data is spread across multiple machines in the cluster, not stored in one place.
`D — Dataset`	It is a collection of records — just like a Python list, but living across many machines.

> **Sateesh Says:** RDD is the backbone of PySpark. Think of it as a magic Python list that lives on 100 computers at once. Whatever you do to one item, Spark automatically does it to all items — across all machines!

Key Properties of RDD

- Immutable — Once you create an RDD, you cannot modify it. Every transformation creates a brand new RDD.
- Lazy Evaluation — Transformations are not executed immediately. They only run when an Action (like collect()) is called.
- Partitioned — Data is split into partitions. Each partition is processed independently on a different machine.
- In-Memory — RDDs try to keep data in RAM for speed. If RAM is full, they spill to disk.

> ⚠ **Key Point:** Transformations (map, flatMap, sortBy...) are LAZY — they just build a plan. Actions (collect, take) are EAGER — they actually execute that plan and return results.

Section 2 — Environment Setup (Google Colab)

Step-by-Step Colab Setup

We use Google Colab because it gives us a free cloud machine with internet access. PySpark needs Java and Spark to be installed. Here is the complete setup you ran in class:

```
# Google Colab PySpark Installation

# Step 1: Install Java (PySpark needs Java to run)
!sudo apt update
!apt-get install openjdk-8-jdk-headless -qq > /dev/null

# Step 2: Download and extract Spark
!wget -q https://dlcdn.apache.org/spark/spark-3.2.1/spark-3.2.1-bin-hadoop3.2.tgz
!tar xf spark-3.2.1-bin-hadoop3.2.tgz

# Step 3: Install Python packages
!pip install -q findspark
!pip install pyspark
!pip install py4j

# Step 4: Initialise findspark so Python can find Spark
import findspark
findspark.init()
findspark.find()
```

Creating a SparkSession

SparkSession is the entry point to PySpark. Think of it as the 'main office' of your Spark application. Everything — reading data, creating RDDs, running jobs — goes through SparkSession.

```
# SparkSession Setup

from pyspark.sql import SparkSession

# Create a SparkSession (driver program)
spark = SparkSession \
        .builder \
        .appName('Our First Spark Example') \
        .getOrCreate()

# Get the SparkContext — needed to create RDDs
sparkContext = spark.sparkContext

print('Spark Version:', spark.version)
```

Object	What it does
`SparkSession`	Main entry point — used for DataFrames and SQL (modern way)

`SparkContext (sc)`	Older core object — used specifically to create and work with RDDs

> **Sateesh Says:** SparkSession is like the CEO of your Spark application. SparkContext is the factory floor manager. In class we mostly use sparkContext / sc to create RDDs.

Section 3 — Creating an RDD (3 Methods)

PySpark gives you three main ways to create an RDD. We covered all three in class.

Method 1 — parallelize()

parallelize() converts a regular Python list (or any in-memory collection) into an RDD. Spark distributes the list items across partitions automatically.

> **Real-World Analogy:** You have a WhatsApp group message list of 10 Swiggy orders. You hand them out across 3 delivery boys. Each delivery boy works on their batch in parallel. parallelize() is that 'handing out' step.

```
# Method 1: parallelize()

# Simple number list
data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# parallelize() distributes the list across Spark partitions
demo = sparkContext.parallelize(data)

# collect() brings all data back to the driver (your machine)
demo.collect()
```

Output:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

> ⚠ **Key Point:** Use parallelize() when you have a small dataset already in Python memory and want to test/learn with it. Never use parallelize() on very large datasets in production.

Method 2 — textFile()

textFile() reads a text file from disk (local, HDFS, or S3) and creates an RDD. Each line of the file becomes one element in the RDD.

> **Real-World Analogy:** Think of a CSV file as a notebook with one customer record per line. textFile() reads each line and hands it to a Spark worker — just like assigning one page of the notebook to each team member.

```
# Method 2: textFile()

# Create an RDD from a CSV file — each line = one RDD element
rdd1 = sparkContext.textFile('/content/sample_data/california_housing_test.csv')

# take(2) fetches only the first 2 elements (header + first row)
rdd1.take(2)
```

Output:

```
['longitude,latitude,housing_median_age,...',
 '-122.05,37.37,27.0,...']
```

Method 3 — range()

range() creates an RDD of integers, just like Python's built-in range() function, but the result is a distributed RDD across Spark partitions.

```
# Method 3: range()

# Create RDD with even numbers from 5 to 70
# start=5, end=70, step=2 (every 2 numbers)
rdd2 = sparkContext.range(start=5, end=70, step=2)

# take(20) fetches the first 20 elements
rdd2.take(20)
```

Output:

```
[5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35, 37, 39, 41, 43]
```

Quick Comparison — When to Use Which?

Method	Best For	Example Use Case
`parallelize()`	In-memory Python list → RDD	Testing logic with small sample data in Colab
`textFile()`	Reading files from disk/cloud	Processing CSV logs from S3 or HDFS
`range()`	Generating a number sequence	Creating test IDs, index generation, benchmarks

Sateesh Says: In real Infosys or TCS projects, you will mostly use textFile() or DataFrame readers. But parallelize() is your best friend when learning — build small, test fast, then scale!

Section 4 — map() Transformation

What is map()?

map() applies a function to every single element in the RDD and returns a new RDD with exactly the same number of elements — but transformed.

> **Factory Analogy:** Think of an Amazon warehouse assembly line. 10 raw boxes go in, each gets a barcode sticker applied, 10 labelled boxes come out. One item in = one item out, but changed. That is exactly what map() does.

Syntax: `rdd.map(lambda x: <transformation>)`

Example 1 — Multiply Numbers by 5

```
# map() — Multiply Numbers

numbers = [1, 2, 3, 4, 5]
numberrdd = sparkContext.parallelize(numbers)

# Multiply each number by 5
# lambda x: x*5  —  x takes each value: 1, 2, 3, 4, 5
maprdd = numberrdd.map(lambda x: x * 5)
maprdd.take(2)
```

Output:

```
[5, 10]
```

Example 2 — Add Suffix '-ly' to Words

In this example, we add 'ly' to the end of each word — transforming adjectives into adverbs. Same number of items in and out.

```
# map() — String Transformation

words = ['quick', 'bright', 'silent']
wordrdd = sparkContext.parallelize(words)

# Append 'ly' to the end of every word
rdd2 = wordrdd.map(lambda x: x + 'ly')
rdd2.collect()
```

Output:

```
['quickly', 'brightly', 'silently']
```

Example 3 — Double Each Number

```
# map() — Double Numbers
```

```
numbers = [1, 2, 3, 4, 5]
numrdd = sparkContext.parallelize(numbers)

# Double every number — classic map() demo
doubled_numbers = numrdd.map(lambda x: x * 2)
doubled_numbers.take(2)
```

Output:

```
[2, 4]
```

Mental Model — map() Visualised

```
Input  RDD:  [1,  2,  3,  4,  5]
              |   |   |   |   |
map(x*2):    x2  x2  x2  x2  x2
              |   |   |   |   |
Output RDD:  [2,  4,  6,  8,  10]

Rule: len(output_rdd) == len(input_rdd)  <--- ALWAYS
```

> **Sateesh Says:** map() is like Flipkart adding a 'discount price' column to every product in their catalog. For every item that goes in, one enhanced item comes out. 10 products in, 10 products out — each with a new field.

Section 5 — flatMap() Transformation

What is flatMap()?

flatMap() is like map() with a superpower — it can produce zero, one, or MANY outputs per input element, and then it flattens all those outputs into a single flat list.

> **Courier Bag Analogy:** Imagine each Flipkart order bag can have multiple items. map() would give you a list of bags. flatMap() opens every bag, takes all items out, and places them in one single flat pile. That pile = your flatMap result.

Syntax: `rdd.flatMap(lambda x: <list of outputs>)`

Example 1 — Split Sentences into Words

Here each sentence (1 input) splits into multiple words (many outputs). All words from all sentences land in ONE flat list.

```
# flatMap() — Split Sentences

sentences = ['Hello world', 'Spark is great', 'Map and flatMap']
rdd4 = sparkContext.parallelize(sentences)

# Split each sentence into words — each sentence gives MULTIPLE words
# flatMap flattens all word lists into a single flat RDD
characters = rdd4.flatMap(lambda x: x.split(' '))
characters.collect()
```

Output:

```
['Hello', 'world', 'Spark', 'is', 'great', 'Map', 'and', 'flatMap']
```

Example 2 — Extract Individual Characters from Words

Here each word is exploded into its individual characters. 'apple' becomes ['a','p','p','l','e'].

```
# flatMap() — Character Extraction

words = ['apple', 'banana', 'grape']
rddwords = sparkContext.parallelize(words)

# list('apple') = ['a','p','p','l','e']
# flatMap opens every word-list and flattens all chars together
characters = rddwords.flatMap(lambda x: list(x))
characters.collect()
```

Output:

```
['a','p','p','l','e','b','a','n','a','n','a','g','r','a','p','e']
```

map() vs flatMap() — The Critical Difference

```
rdd = sc.parallelize(['Hello World', 'PySpark Rocks'])

# map() — each element stays as a LIST
rdd.map(lambda x: x.split(' ')).collect()
# => [['Hello', 'World'], ['PySpark', 'Rocks']]   <-- list of lists!

# flatMap() — all results merged into ONE flat list
rdd.flatMap(lambda x: x.split(' ')).collect()
# => ['Hello', 'World', 'PySpark', 'Rocks']       <-- single flat list!
```

Feature	map()	flatMap()
Output per input	Exactly 1 item	0, 1, or MANY items
Result shape	Same length as input	Typically longer than input
When to use	Enrich / reshape records	Explode / flatten nested data
Common use	Add a new column to each row	Extract words, tags, items from text

Sateesh Says: When Zomato's search engine indexes a restaurant's menu, it does exactly flatMap() — one menu (1 input) explodes into hundreds of individual dish keywords (many outputs). That is the real-world power of flatMap().

Section 6 — reduceByKey() Transformation

What is reduceByKey()?

reduceByKey() works ONLY on key-value pair RDDs — that is, RDDs where each element is a tuple (key, value). It groups all values with the same key and combines them using a function you provide.

> **Kirana Store Analogy:** Imagine a Kirana store owner collecting all daily sales slips. He groups them by product: apple slips in one pile, banana slips in another. Then he adds up all amounts in each pile. That grouping = grouping by key. That adding = the reduce function. Together: reduceByKey!

Syntax: `pair_rdd.reduceByKey(lambda a, b: a + b)`

Example 1 — Total Sales by Product (Fruit Store)

```
# reduceByKey() — Fruit Sales

# Each tuple: (product name, sale amount)
sales = [('apple',100),('banana',200),('apple',150),('orange',300),('banana',50)]

rdd = sc.parallelize(sales)

# reduceByKey groups by product name and sums amounts
# lambda a, b: a + b  -->  adds amounts with the same product key
total_sales_rdd = rdd.reduceByKey(lambda a, b: a + b)
total_sales_rdd.collect()
```

Output:

```
[('apple', 250), ('banana', 250), ('orange', 300)]
```

apple: 100 + 150 = 250 | banana: 200 + 50 = 250 | orange: 300 (only one)

Example 2 — Word Count (Classic Big Data Problem)

Word count is the 'Hello World' of Big Data. Count how many times each word appears in a list.

```
# reduceByKey() — Word Count

words = ['apple', 'banana', 'apple', 'orange', 'banana', 'apple']

rdd = sc.parallelize(words)

# Step 1: map() — convert each word to (word, 1) pair
pairs_rdd = rdd.map(lambda word: (word, 1))
# pairs_rdd =
[('apple',1),('banana',1),('apple',1),('orange',1),('banana',1),('apple',1)]

# Step 2: reduceByKey() — add up all 1s for the same word
word_counts_rdd = pairs_rdd.reduceByKey(lambda a, b: a + b)
```

```
word_counts_rdd.collect()
```

Output:

```
[('apple', 3), ('banana', 2), ('orange', 1)]
```

How reduceByKey Works Internally

```
Input pairs:    [('apple',1), ('banana',1), ('apple',1), ('banana',1), ('apple',1)]

After grouping:  apple  ->  [1, 1, 1]
                 banana ->  [1, 1]

After reducing:  apple  ->  1 + 1 + 1 = 3
                 banana ->  1 + 1     = 2

Output:         [('apple', 3), ('banana', 2)]
```

⚠ **Key Point:** reduceByKey() requires a (key, value) pair RDD. Always use map() first to create the (K, V) structure, then apply reduceByKey().

□ **Sateesh Says:** Every time HDFC Bank shows you 'Total Spent This Month — Rs. 18,450', they ran a reduceByKey on your (account_id, amount) transaction records. You just learned one of banking's most used operations!

Section 7 — sortByKey() Transformation

What is sortByKey()?

sortByKey() sorts a key-value pair RDD based on the KEY of each tuple. It returns a new RDD with the same elements, but ordered by key either ascending or descending.

> **School Register Analogy:** Imagine a school teacher has student score slips scattered randomly on the table: ('Priya', 90), ('Rahul', 75), ('Amit', 85). sortByKey() arranges them in alphabetical order by student name — just like sorting a register by roll number or name.

Syntax: `pair_rdd.sortByKey(ascending=True)` or `sortByKey(ascending=False)`

Example 1 — Sort Students by Name (Ascending)

```
# sortByKey() — Student Scores by Name

student_scores = [('John', 88), ('Alice', 95), ('Bob', 78), ('Diana', 85)]

rdd = sparkContext.parallelize(student_scores)

# sortByKey sorts by the FIRST element (name) alphabetically
sort_rdd = rdd.sortByKey(ascending=False)   # Z to A
sorted_scores = sort_rdd.collect()
print(sorted_scores)
```

Output:

```
[('John', 88), ('Diana', 85), ('Bob', 78), ('Alice', 95)]
```

Note: sorted by name in reverse alphabetical order (Z → A): John, Diana, Bob, Alice

Example 2 — Sort Products by Name

```
# sortByKey() — Product Sales

sales_data = [('apple', 150), ('banana', 200), ('orange', 300), ('grape', 100)]

rdd = sparkContext.parallelize(sales_data)

# Sort by product name (key) in descending order (Z to A)
sorted_rdd = rdd.sortByKey(ascending=False)
print(sorted_rdd.collect())
```

Output:

```
[('orange', 300), ('grape', 100), ('banana', 200), ('apple', 150)]
```

Sorted by product name descending: orange > grape > banana > apple

⚠ **Key Point:** sortByKey() ONLY sorts by the key (first element of the tuple). If you want to sort by the value or any other field, use sortBy() instead (covered next).

□ **Sateesh Says:** Think of sortByKey() as sorting a Flipkart product catalogue alphabetically. The product name is the key, and the price is the value. sortByKey() cares only about the name, not the price.

Section 8 — sortBy() Transformation

What is sortBy()?

sortBy() is more flexible than sortByKey(). It works on ANY RDD (not just key-value pairs) and lets you sort by any derived value — you provide a function that tells Spark what to sort by.

> **Marks List Analogy:** A teacher has student records: [('John', 88), ('Alice', 95), ('Bob', 78)]. sortByKey() sorts by name. But sortBy() lets you say 'sort by the SCORE' — the second element. You tell it the rule; Spark does the sorting.

Syntax: `rdd.sortBy(lambda x: x[1], ascending=False)`

Example 1 — Sort Students by Score (Descending)

```
# sortBy() — Sort by Score

student_scores = [('John', 88), ('Alice', 95), ('Bob', 78), ('Diana', 85)]

rdd = sparkContext.parallelize(student_scores)

# sortBy — sort by the SECOND element (score) in descending order
# lambda x: x[1]  =>  x = ('John', 88), x[1] = 88
sorted_rdd = rdd.sortBy(lambda x: x[1], ascending=False)
sorted_scores = sorted_rdd.collect()
print(sorted_scores)
```

Output:

```
[('Alice', 95), ('John', 88), ('Diana', 85), ('Bob', 78)]
```

Sorted by score descending: Alice(95) > John(88) > Diana(85) > Bob(78)

Example 2 — Sort Words by Length

Sort a list of fruit names by their LENGTH (number of characters), longest first.

```
# sortBy() — Sort by Word Length

words = ['banana', 'apple', 'grape', 'pineapple', 'orange']

rdd = sparkContext.parallelize(words)

# sortBy using len(x) as the sort key — word length
sorted_rdd = rdd.sortBy(lambda x: len(x), ascending=False)
sorted_words = sorted_rdd.collect()
print(sorted_words)
```

Output:

```
['pineapple', 'banana', 'orange', 'apple', 'grape']
```

pineapple(9) > banana(6) = orange(6) > apple(5) = grape(5)

sortByKey() vs sortBy() — Final Comparison

Feature	sortByKey()	sortBy()
Works on	Only (K, V) pair RDDs	Any RDD — pairs or plain
Sort by	Always by the KEY	Any value via lambda function
Flexibility	Low — key only	High — sort by anything
Example	Sort cities alphabetically	Sort by score, length, amount
Syntax	rdd.sortByKey(asc=True)	rdd.sortBy(lambda x: x[1])

Sateesh Says: sortByKey() is the express train — goes only to one destination (the key). sortBy() is the cab — you tell the driver exactly where to go. In interviews, they will ask 'what is the difference?' — now you know!

Section 9 — Actions: collect() and take()

Transformations vs Actions — The Fundamental Difference

In PySpark, there are two types of operations on an RDD:

Type	Description
`Transformation`	Builds a NEW RDD from an existing one. LAZY — nothing runs until an Action is called. Examples: map(), flatMap(), sortBy(), reduceByKey()
`Action`	Triggers actual execution of all pending transformations and returns a result. EAGER — runs immediately. Examples: collect(), take(), count()

Recipe vs Cooking Analogy: A Transformation is writing a recipe. An Action is actually cooking the dish. You can write 10 recipe steps (transformations) — nothing gets cooked. The moment you say 'start cooking' (action), Spark executes ALL the steps in one go.

collect() — Bring ALL Data to Driver

collect() is an Action that retrieves every element of the RDD and returns it as a regular Python list on your machine (the driver).

```
# collect() — Action

numbers = [1, 2, 3, 4, 5]
rdd = sparkContext.parallelize(numbers)
doubled = rdd.map(lambda x: x * 2)    # <-- LAZY: nothing runs yet

# collect() triggers execution and returns the full list
result = doubled.collect()            # <-- EAGER: now it runs!
print(result)
```

Output:

```
[2, 4, 6, 8, 10]
```

Key Point: NEVER use collect() on a large dataset in production! If your RDD has 10 crore rows, collect() will try to load all 10 crore records into your driver machine's memory and crash it. Use take() instead.

take(n) — Fetch Only First n Elements

take(n) is a safer Action. It retrieves only the first n elements of the RDD without pulling everything into memory. Perfect for previewing large datasets.

```
# take() — Action

rdd2 = sparkContext.range(start=5, end=70, step=2)

# take(20) fetches only the first 20 numbers — safe on big data
rdd2.take(20)
```

Output:

```
[5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35, 37, 39, 41, 43]
```

Action	When to Use
`collect()`	Only when you are sure the dataset is small. Learning & testing. NEVER on production big data.
`take(n)`	Preview first n rows of any dataset — safe even on billions of records. Use in production.

Sateesh Says: In your interviews at TCS or Wipro, they will always ask: 'What happens if you call collect() on a 1TB RDD?' Answer: It crashes the driver. Always use take() to preview. This shows you understand production realities!

Section 10 — Full Pipeline Demo (Sales Data)

At the end of class, we brought everything together in a single real-world pipeline using the sales dataset. This shows how map, flatMap, reduceByKey, sortByKey, and sortBy chain together.

Dataset — E-Commerce Sales Records

```
# Complete Dataset

# (customer_id, product_category, quantity, price_per_unit)
sales_data = [
    ('C001', 'Electronics', 2, 599.99),
    ('C002', 'Books',       5,  15.50),
    ('C001', 'Clothing',    1,  79.99),
    ('C003', 'Electronics', 1,1299.99),
    ('C002', 'Electronics', 3, 299.99),
    ('C004', 'Books',       2,  25.00),
    ('C003', 'Clothing',    4,  45.99),
    ('C001', 'Books',       8,  12.99),
    ('C004', 'Electronics', 1, 899.99),
    ('C002', 'Clothing',    2,  89.99)
]
sales_rdd = sc.parallelize(sales_data)
```

Pipeline Step 1 — map() to Calculate Total Amount

```
# Step 1: map()

# Add total_amount = quantity * price_per_unit as a new field
amount_rdd = sales_rdd.map(lambda x: (x[0], x[1], x[2]*x[3]))
amount_rdd.take(3)
```

Output:

```
[('C001', 'Electronics', 1199.98),
 ('C002', 'Books', 77.5),
 ('C001', 'Clothing', 79.99)]
```

Pipeline Step 2 — flatMap() to Tokenise Categories

```
# Step 2: flatMap()

# Extract category as individual tokens (splits multi-word categories)
category_tokens = sales_rdd.flatMap(lambda x: x[1].lower().split(' '))
category_tokens.collect()
```

Output:

```
['electronics', 'books', 'clothing', 'electronics', 'electronics', ...]
```

Pipeline Step 3 — reduceByKey() for Category Revenue

```
# Step 3: reduceByKey()

# (category, quantity*price) then sum by category
cat_rdd   = sales_rdd.map(lambda x: (x[1], x[2]*x[3]))
cat_total = cat_rdd.reduceByKey(lambda a, b: a + b)
cat_total.collect()
```

Output:

```
[('Electronics', 3889.94), ('Books', 207.5), ('Clothing', 395.95)]
```

Pipeline Step 4 — sortByKey() Alphabetical Report

```
# Step 4: sortByKey()

# Sort category totals alphabetically by category name
sorted_cat = cat_total.sortByKey(ascending=True)
sorted_cat.collect()
```

Output:

```
[('Books', 207.5), ('Clothing', 395.95), ('Electronics', 3889.94)]
```

Pipeline Step 5 — sortBy() Top Spending Customer

```
# Step 5: sortBy()

# Customer-level total spend
cust_rdd   = sales_rdd.map(lambda x: (x[0], x[2]*x[3]))
cust_total = cust_rdd.reduceByKey(lambda a, b: a + b)

# Sort by spend value (2nd element) — highest first
ranked = cust_total.sortBy(lambda x: x[1], ascending=False)
ranked.collect()
```

Output:

```
[('C003', 1483.95), ('C004', 949.99), ('C001', 1383.97), ('C002', 1277.44)]
```

Sateesh Says: This 5-step pipeline is what a real Swiggy analyst builds at 2 AM before the weekly business review presentation. You just learned production-level PySpark in one session!

Section 11 — Session Recap & RDD Cheat Sheet

What We Covered Today

1. What is an RDD — Resilient, Distributed, Dataset. The backbone of PySpark.
2. 3 Ways to Create an RDD — parallelize(), textFile(), range()
3. map() — 1 input = 1 output. Transforms / enriches each record.
4. flatMap() — 1 input = 0, 1, or many outputs. Flattens nested data.
5. reduceByKey() — Groups by key and aggregates values. Works on (K,V) pairs.
6. sortByKey() — Sorts a (K,V) RDD by KEY. Alphabetical or numeric.
7. sortBy() — Sorts ANY RDD by a derived value via lambda. More flexible.
8. collect() — Action: brings all data to driver. Use only on small data.
9. take(n) — Action: fetches first n elements safely. Use in production.

Complete RDD Operations Cheat Sheet

Operation	Type	Syntax Example
`parallelize()`	Create RDD	sc.parallelize([1,2,3,4,5])
`textFile()`	Create RDD	sc.textFile('/path/to/file.csv')
`range()`	Create RDD	sc.range(start=0, end=100, step=2)
`map()`	Transformation	rdd.map(lambda x: x * 2)
`flatMap()`	Transformation	rdd.flatMap(lambda x: x.split(' '))
`reduceByKey()`	Transformation	rdd.reduceByKey(lambda a,b: a+b)
`sortByKey()`	Transformation	rdd.sortByKey(ascending=True)
`sortBy()`	Transformation	rdd.sortBy(lambda x: x[1], ascending=False)
`collect()`	Action	rdd.collect() # returns Python list
`take(n)`	Action	rdd.take(5) # returns first 5 items

The Golden Rule — Lazy vs Eager

```
rdd = sc.parallelize([1,2,3,4,5])

step1 = rdd.map(lambda x: x * 2)          # LAZY — builds a plan
step2 = step1.map(lambda x: x + 1)         # LAZY — adds to the plan
step3 = step2.sortBy(lambda x: x)          # LAZY — adds to the plan

# Nothing has run yet. Spark is just making a recipe.

result = step3.collect()   # EAGER — NOW Spark executes ALL 3 steps
print(result)
# => [3, 5, 7, 9, 11]
```

⚠ **Key Point:** Spark optimises the execution plan during lazy evaluation. This is why chaining transformations is efficient — Spark can combine multiple steps into one pass through the data.

Common Interview Questions from This Session

- What does the 'R' in RDD stand for, and what does it mean practically?
- What is the difference between map() and flatMap()?
- What is the difference between sortByKey() and sortBy()?
- Why is lazy evaluation beneficial in PySpark?
- When would you use take() instead of collect()?
- What type of RDD does reduceByKey() require?
- How do you create an RDD from an existing Python list?